

Troubleshooting Matrix

Trainee Misconception

1. State as Global Variables
Trainees store intermediate variables, token counters, or DB connections in globals and expect persistence across interruptions.

2. Forgetting Reducers
Trainees add list fields such as audit_trail without an explicit reducer and assume new items append automatically.

3. Non-Serializable State
Trainees place live DB sessions, HTTP clients, or mock objects inside workflow state.

4. Bypassing State Guards
When a transition fails, trainees comment out can_transition() instead of fixing the missing workflow step, such as task assignment.

5. Cross-Provider Config Leaks
Trainees hardcode provider-specific parameters, such as OpenAI native client options, inside agent calls.

Why It Breaks

Checkpointers save only values explicitly returned in the state dict. Globals vanish when a run resumes in a new process.

LangGraph wholesale-overwrites state keys by default, silently wiping prior audit records.

PostgresSaver-style checkpointers serialize with msgpack or JSON; runtime objects trigger a fatal TypeError.

Removing guards creates security and domain-logic vulnerabilities by letting review tasks skip lifecycle gates, such as pending → approved.

Switching via LLM_PROVIDER=ollama makes unsupported provider-specific arguments fail at runtime.

Instructor Correction & Solution

“If it’s not in the state TypedDict, it won’t survive a pause/resume cycle.” Return durable values at checkpoint boundaries.

Point to Part 3, Exercise A. Use Annotated[list, operator.add] so the state engine concatenates instead of overwriting.

Keep state data-driven: primitives, plain dicts, or serializable dataclasses. Use constructor injection for LLM and repository dependencies outside the graph.

Point to Part 3, Exercise C. Preserve guards as the legal defense of domain invariants; fix the upstream step.

Point to Part 3, Exercise D. Use abstract uniform kwargs; OpenAIAdapter and OllamaAdapter translate them explicitly.